2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS) | 979-8-3503-3071-7/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICPADS60453.2023.00415

Accelerating Distributed Training through In-network Aggregation with Idle Resources in Data Centers

Ge Wang[1] Huaqing Tu[2] Gongming Zhao[1] Hongli Xu[1]
[1]University of Science and Technology of China, [2]Zhejiang Lab

***Abstract*—As the parameter scale of large-scale models continues to increase, distributed model training imposes significant communication overhead in data centers, resulting in reduced training efficiency. To address this challenge, a promising solution, called in-network aggregation, is proposed to mitigate the bandwidth bottleneck in data centers by aggregating gradients within network. However, existing works mainly rely on programmable switches to implement in-network aggregation, while programmable switches have limited memory resources, which makes it hard to storage parameter with large size. Moreover, programmable switches are not yet widely deployed in current data centers, resulting in poor availability. To overcome this limitation, we leverage servers with idle resources in data centers for in-network aggregation, since servers have more powerful storage capabilities compared with programmable switches. Specifically, we formally formulate the problem of server-based in-network aggregation. An efficient approximate algorithm with bounded approximation factor is proposed to select servers with idle resources and paths for model aggregation. Our extensive simulations show that our proposed method can reduce communication time by 38.4%-60.1% compared to state-of-the-art solutions.**

***Index Terms*—In-network Aggregation, Distributed Model Training, Data Center Network**

I. Introduction

The emerging deep learning [1] is a powerful tool for data analysis, which plays a significant role in many fields, (*e.g.*, image recognition, content recommendation and natural language processing). Distributed model training has emerged as a promising solution for its ability to handle larger amounts of data and parameters, and its potential to improve both training capacity and efficiency. This approach allows for the parallelization of training across multiple *workers*, each of which can be a GPU, a machine, or a training entity, enabling faster and more scalable training. In architecture, these workers are connected to PS via switches to from a network, typically within a datacenter, which enhances training efficiency.

To converge and optimize the training model, distributed model training usually consists of multiple iterations in parameter server (PS) architecture. At the end of each iteration, each worker needs to compute a local model and send its training results (parameters, models) to PS for aggregation. As the scale of distributed model training continues to grow, especially with the emergence of large language model (LLM) [2], more workers are required for the model training and larger local model are used. The performance bottleneck of training has been shifted from computing to communication [3], [4]. To reduce communication overhead, an effective way is reducing the amount of information transmitted, such as compressing the model that needs to be transmitted or reducing the number of workers send the model to PS. The authors in [5] reduce the communication bandwidth by sending only the compressed model, that is, only parameters larger than a threshold are transmitted. Matcha [6] selects a partial of workers which need to send the local model to the parameter server in one iteration. However, model compression and worker selection cause training performance degradation. Moreover, model compression also increase the computation pressure of workers.

With the rapid development of programmable network devices (*e.g.*, programmable switches [7], [8]), one promising method called *in-network aggregation* is proposed to reduce the communication overhead. In-network aggregation utilizes programmable switches to aggregate the training results from multiple workers along their ways sent to PS. For example, SwitchML [7] performs in-network aggregation by aggregating local models updated from serviced workers on Top of Rack (ToR) switches. ATP [8] performs in-network aggregation on the rack switch at the workers or the PS. However, programmable switches are much more expensive compared with traditional switches, making it difficult to be deployed on a large scale. This greatly reduces the availability of programmable switches for in-network aggregation in real network scenarios. More importantly, programmable switches have limited memory resources (about O(10MB) [9]), so it is hard to aggregate parameters with large size. Therefore, there is an urgent need to find network devices which are more suitable than programmable switches for in-network aggregation.

We observe that parameters sent from workers to PS will pass through not only switches but also many servers in a network. In current data centers which often are software-defined, servers may perform as a middle-box or a network function. For example, in some data center networks, servers often assume a portion of the switch functions for forwarding network flows [10], [11]. This makes it possible to use servers along with the routing path for in-network aggregation. Actually, servers are advantageous over programmable switches for in-network aggregation in two aspects. On one hand, servers are common in data centers while programmable switches are not. On the other hand, a server have O(100GB) of storage space, which is far more than that of a programmable switch. Thus, memory resources will not be a bottleneck for in-network aggregation.

Based on the above observations, in this paper, we propose

2690-5965/23/$31.00 ©2023 IEEE
DOI 10.1109/ICPADS60453.2023.00415

Authorized licensed use limited to: Zhejiang Lab. Downloaded on August 03,2025 at 03:31:12 UTC from IEEE Xplore. Restrictions apply.

utilizing the idle resources of servers for in-network aggregation to reduce the communication cost in distributed model training. Since servers are used for aggregation in the routing path of transmitting parameters from workers to PS, it will not interfere with the training of model task, so the model training is not aware of in-network aggregation. When using servers to enable in-network aggregation, the following problem should be addressed. First, how to select a suitable route path for each local training model to be sent to the server, subject to the constraint of link capacity; Second, how to decide where the local models from workers are aggregated along their routing paths to PS; Third, how to ensure that the constraints of server resources, such as processing capacity and memory capacity, are not violated. To cope with the above challenges, this paper designs an efficient approximate algorithm and evaluate its performance with real dateset.

II. System Model and Problem Formulation

A. System Model

*1) **Distributed Model training**:* In distributed model training, each worker $w_i \in W$ updates the local model parameter ψ_i based on mini-batch sampled from local dataset, where $W = \{w_1, w_2, \ldots, w_{|W|}\}$ denotes the worker set. In terms of the parallelism schemes, the distributed model training can be categorized into two types: model parallelism and data parallelism. This paper focuses on the data parallelism of distributed model training, which splits the whole dataset into multiple compute nodes and uses the same model on each worker, *i.e.*, $|\psi_i| = |\psi|, w_i \in W$ and ψ is the global model. Local models need to be updated on the local dataset and then aggregated to the parameter server through hundreds of iterations to achieve convergence. Specifically, after each iteration, workers send local models to parameter server. The parameter server updates the global model based on the received models and sends the updated model back to workers. The global model will be updated as follows,

$$\psi = \frac{\sum_{w_i \in W} \psi_i}{|W|} \tag{1}$$

To guarantee the model convergence and considering the heterogeneity of devices, this phase is usually performed synchronously in data centers.

*2) **In-network Aggregation**:* In an in-network aggregation system, besides parameter server and workers, there a set of servers with idle resources. We denote these servers as $S = \{s_1, s_2, \ldots, s_{|S|}\}$. The aggregation operation is not only performed on the parameter server, but also performed on the servers. Different from the aggregation in PS, the aggregation on servers only summarize the received model parameters, that is

$$\psi_s = \sum_{w_i \in W_s} \psi_i \tag{2}$$

where ψ_s is the aggregated model on server s and W_s is the set of workers which send models to server s.

After summary, each server sends the aggregated model to PS or another server. In order to avoid repeated aggregation of the worker's local model, each worker will only send the model to an server or directly to PS. In this way, the aggregation on PS can be represent as follows,

$$\psi = \frac{\sum_{w_i \in W_{PS}} \psi_i + \sum_{s \in S} \psi_s}{|W|} = \frac{\sum_{w_i \in W} \psi_i}{|W|} \tag{3}$$

where W_{PS} denotes the set of workers which send local model to parameter server directly.

Then, we consider the model transmission process of in-network aggregation. In order to ensure that the model can be transmitted and computed in a timely manner, the transmission and aggregation of each model require a certain amount of bandwidth and computational resources. Considering that in data parallelism, the local model ψ_i on each worker w_i are of the same size, the bandwidth requirement of sending each model is denoted as b and the aggregation capacity requirement is denoted as c. Additionally, the bandwidth of the PS is denoted as B_{PS}. For each server s, we denote its idle bandwidth as B_s and idle aggregation computing capacity as C_s.

In data centers, the network virtual topology comprising parameter server, servers, and workers can be depicted as an undirected complete graph $G = (V, E)$, where $V = \{W \cup S \cup PS\}$ is the set of all node components and E denotes the set of links which connect node pairs. Taking into account various factors, such as the number of switch hops required, the physical distance between nodes, and the inherent quality of the links, the cost associated with each link differs. Specifically, we use c_e to represent the path cost of each edge $e \in E$.

B. Problem Formulation

We use binary variable $x_e \in \{0, 1\}$ to denote whether edge $e \in E$ is selected as the link for model aggregation ($x_e = 1$) or not ($x_e = 0$). In addition, in order to formalize the in-network aggregation problem, we need to define two additional functions. For a subset $H \subset V$, $\delta(H)$ denotes the set of edges with exactly one endpoint in H. Set $f(H) = \max_{u \in H, v \notin H} \{r_{u,v}\}$ for each subset $H \subset V$, where $r_{u,v} \in \{0, 1\}$ denotes whether the pair of vertices $(u, v) \in V$ has connectivity requirements or not. In our problem, each worker $w \in W$ has connectivity requirements with PS, *i.e.*, $r_{w,PS} = 1$. Thus, $f(H) = 1, \forall H \subset W$. The weakly supermodular function f on the sets of vertices is defined by Jain's method [12] which specify the connectivity requirement. In particular, for a vertex $v \in V$, $\delta(v)$ denotes the set of edges incident to v.

Furthermore, we consider the resource utilization on each server. Based on our analysis in the previous section, each server will have only one connection for sending, while the remaining connections are used for receiving models from different workers. Thus, the number of local models received on each server is $(\sum_{e \in \delta(s)} x_e - 1)$. The number of computations required to complete the summation of models is $(\sum_{e \in \delta(s)} x_e - 1) - 1 = (\sum_{e \in \delta(s)} x_e - 2)$. Therefore, it is essential to identify the optimal path for each worker, with the

Authorized licensed use limited to: Zhejiang Lab. Downloaded on August 03,2025 at 03:31:12 UTC from IEEE Xplore. Restrictions apply.

objective of minimizing the overall link cost, thereby achieving the most efficient model transmission performance.

III. Algorithm Design

We propose an iterative rounding algorithm for the BDTST problem, as described in Alg.1. The BDTST algorithm mainly consists of two steps. In the first step, we formulate the BDTST problem as an integer programming and a construct linear programming as relaxation of BDTST (LP-BDTST). We first initialize the solution subtree $T' = \emptyset$; and connectivity requirements function $f'(H) = f(H)$ for $\forall H \subset W$. Moreover, considering that the edges between two workers are improper in a practical solution. We remove all edges $e = (u, v)$ from graph G where $u, v \in W$. Since LP-BDTST is a linear programming, we can obtain a basic optimum solution x by a convex programming solver, such as Gurobi solver.

After an iteration of rounding, we obtain a basic Steiner tree solution and remove every edge e with $x_e = 0$. If there is a vertex $v \in V$ with degree at most $Bdev(v) + 3$, the BDTST algorithm removes the degree constraint for that vertex. This relaxation step only incurs an additive constant 3 on the degree bounds. Then, BDTST adds every edge e with $x_e = 1$ to the basic subtree T' (initially T' is empty) and decreases the degree bounds of two endpoints by 1, respectively. In *step 1(d)*, each heavy edge with fractional value $x_e \geq \frac{1}{2}$ is picked only if both endpoints do not have degree constraints. The last step in an iteration is to update the connectivity requirements, *i.e.*, set $f'(S) = f(S) - \delta_{T'}(H)$. Then consider the residual problem, where the edges in T' are pre-selected, and repeat the above procedure until all the connectivity requirements are satisfied. Finally, the first step of BDTST algorithm returns a basic Steiner tree T'.

In the second step, we will construct a Terminal Steiner tree from the basic Steiner tree obtained in the first step. The weight of the basic Steiner tree T' is at most $2d(T^*)$ such that $deg(v) \leq 4$ for each $v \in W$ and $deg(v) \leq Bdeg(v) + 3$ for each $v \in V \setminus W$. Observe that T' is not a Terminal Steiner tree since some vertices $v \in W$ may not be leaves. Let the neighbors of w in T' be $v_1, v_2, \ldots, v_k$, $k \geq 2$ and $v_i \in V \setminus W$ for each $1 \leq i \leq k$. Let (w, v_k) be the edge incident at w in T' with largest path cost. Then, we replace the edges incident at w by edges $\{(w, v_1), (v_1, v_2), \ldots, (v_{k-1}, v_k)\}$. Finally, the BDTST algorithm returns a Terminal Steiner tree T with degree bounds guarantee.

Algorithm 1 BDTST: Bounded-Degree Terminal Steiner Tree Algorithm

Step 1: Solve the LP-BDTST Problem.
Initialization: $T' = \emptyset$, $f'(H) = f(H)$ for $\forall H \subset W$. Remove all edges $e = (u, v)$ from graph G where $u, v \in W$.
while T' is not a Steiner tree **do**
  (a) Computing a basic optimal solution:
  Find a basic optimal solution x satisfying f' and remove every edge e with $x_e = 0$.
  (b) Removing a degree constraint:
  For every $v \in V$ with degree at most $Bdeg(v) + 3$, remove v from V.
  (c) Picking an 1-edge:
  For each edge $e = \{u, v\}$ with $x_e = 1$, add e to T', remove e from G and decrease $Bdeg(u)$, $Bdeg(v)$ by 1, respectively.
  (d) Picking the heavy edges with no degree constraints:
  For each edge $e = \{u, v\}$ with $x_e \geq \frac{1}{2}$ and $u, v \notin V$, add e to T' and remove e from G.
  (e) Updating the connectivity requirements:
  Set $f'(H) = f(H) - |\delta_{T'}(H)|$.
end while
Return a basic solution Steiner tree T'.
Step 2: Construct a Terminal Steiner Tree.
for Each worker $w \in W$ **do**
  if $deg(w) > 1$ **then**
    (a) Let the neighbors of w in T' be $v_1, v_2, \ldots, v_k$, where $k \geq 2$ and $v_i \in V \setminus W$ for each $1 \leq i \leq k$.
    (b) Let (w, v_k) be the edge incident at w in T' with largest path cost.
    (c) Replace the edges incident at w by edges $\{(w, v_1), (v_1, v_2), \ldots, (v_{k-1}, v_k)\}$.
  end if
end for
Return a Terminal Steiner tree T.

IV. Experiments

A. Experimental Setup

We employed containers to simulate workers, servers with idle resources, and parameter servers. Drawing inspiration from the c-Through topology [10], we constructed a Fat-Tree topology, in which a portion of the ToR (Top-of-Rack) switch functionality within the Fat-Tree topology is undertaken by the servers. More specifically, we deployed four workers and one server on each of the four physical servers equipped with NVIDIA graphics cards. The server served the role of a ToR switch. Parameter server containers were randomly deployed on a physical server. The bandwidth of the servers are set between 100 to 1000 MB/s randomly and the bandwidth of parameter server is set as 1000 MB/s. We choose two popular benchmark datasets with models for performance evaluation: AlexNet model on EMNSIT dataset and VGG16 mode on ImageNet dataset.

B. Metrics and Benchmarks

*1) **Metrics**:* We adopt the following performance metrics for performance comparison. (1) *Training time.* We record the time between two consecutive iterations until achieve expected accuracy. (2) *Communication time per iteration.* In each iteration, we measure the duration starting from the time that workers sending local models to server os PS till all models are aggregated by PS as the communication time per iteration.

Authorized licensed use limited to: Zhejiang Lab. Downloaded on August 03,2025 at 03:31:12 UTC from IEEE Xplore. Restrictions apply.

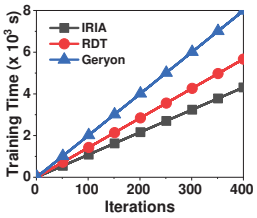

Fig. 1: Training Time vs. Iterations on EMNIST.

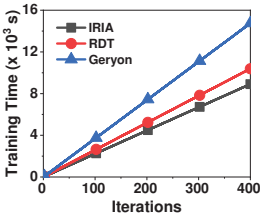

Fig. 2: Training Time vs. Iterations on ImageNet.

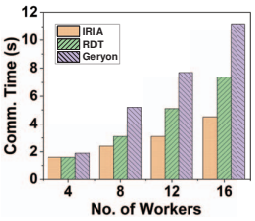

Fig. 3: Communication time per iteration vs No. workers on EMNIST.

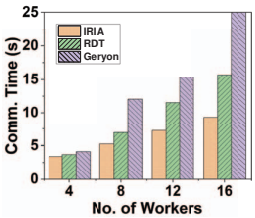

Fig. 4: Communication time per iteration vs No. workers on ImageNet.

*2) **Banchmarks**:* We adopt two benchmarks to compare with our algorithm. (1) *Geryon* [13] represents the distributed training scheduling without considering in-network aggregation. Geryon selects the shortest path to the PS for each gradient under the resource constraints. (2) *RDT* [14] perform in-network aggregation with fixed routing path. In the original work, aggregation was based on actual geographical locations. In our experiments, we set the path to be the shortest path towards the PS. If there are servers along this path, we perform in-network aggregation. We should note that both of the benchmarks may exceed the resource constraints.

C. Performance Results

From Figures 1-2, it can be observed that our IRIA solution completed the training rounds with the least amount of time. Specifically, Figures 1-2 show that our approach was able to complete the same iterations in the shortest time.

In the third set of experiments, we demonstrated the communication time cost during each iteration of the distributed model training. Figures 3-4 show that the IRIA approach consistently required the least communication time in each iteration. For example, in Figure 3, when the number of workers is 16, the IRIA solution reduces communication time by 39.1% and 59.8% relative to the RDT and Geryon solutions, respectively. This was not only due to our utilization of servers to perform in-network aggregation, which reduced the total amount of data to be transmitted, but also because our algorithm selected the path with the minimum cost while ensuring connectivity.

V. Conclusions

In this paper, we propose a new in-network aggregation solution with idle resources in data centers. Additionally, we develop an efficient algorithm to select servers and paths for model aggregation. Our extensive simulations show that our proposed method can reduce communication time by 38.4%-60.1% compared to state-of-the-art solutions.

Acknowledgement

The corresponding author of this paper is Huaqing Tu.This work was supported in part by the National Science Foundation of China (NSFC) under Grants 62132019, 62102392 and Grant 62372426, in part by the National Science Foundation of Jiangsu Province under Grant BK20210121, in part by the Hefei Municipal Natural Science Foundation under Grant 2022013, in part by the Youth Innovation Promotion Association of Chinese Academy of Sciences under Grant 2023481, and in part by the Fundamental Research Funds for the Central Universities.

References

[1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[2] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[3] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro *et al.*, "Efficient large-scale language model training on gpu clusters using megatron-lm," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021, pp. 1–15.

[4] M. Naumov, J. Kim, D. Mudigere, S. Sridharan, X. Wang, W. Zhao, S. Yilmaz, C. Kim, H. Yuen, M. Ozdal *et al.*, "Deep learning training in facebook data centers: Design of scale-up and scale-out systems," *arXiv preprint arXiv:2003.09518*, 2020.

[5] Y. Lin, S. Han, H. Mao, Y. Wang, and W. J. Dally, "Deep gradient compression: Reducing the communication bandwidth for distributed training," *arXiv preprint arXiv:1712.01887*, 2017.

[6] J. Wang, A. K. Sahu, Z. Yang, G. Joshi, and S. Kar, "Matcha: Speeding up decentralized sgd via matching decomposition sampling," in *2019 Sixth Indian Control Conference (ICC)*. IEEE, 2019, pp. 299–300.

[7] A. Sapio, M. Canini, and C.-Y. Ho, "Scaling distributed machine learning with In-Network aggregation," in *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. USENIX Association, 2021, pp. 785–808.

[8] C. Lao, Y. Le, K. Mahajan, and Chen, "{ATP}: In-network aggregation for multi-tenant learning," in *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, 2021, pp. 741–761.

[9] D. R. Ports and J. Nelson, "When should the network be the computer?" in *Proceedings of the Workshop on Hot Topics in Operating Systems*, 2019, pp. 209–215.

[10] G. Wang, D. G. Andersen, M. Kaminsky, K. Papagiannaki, T. E. Ng, M. Kozuch, and M. Ryan, "c-through: Part-time optics in data centers," in *Proceedings of the ACM SIGCOMM 2010 Conference*, 2010, pp. 327–338.

[11] C. Guo, G. Lu, D. Li, H. Wu, X. Zhang, Y. Shi, C. Tian, Y. Zhang, and S. Lu, "Bcube: a high performance, server-centric network architecture for modular data centers," in *Proceedings of the ACM SIGCOMM 2009 conference on Data communication*, 2009, pp. 63–74.

[12] K. Jain, "A factor 2 approximation algorithm for the generalized steiner network problem," *Combinatorica*, vol. 21, pp. 39–60, 2001.

[13] S. Wang, D. Li, and J. Geng, "Geryon: Accelerating distributed cnn training by network-level flow scheduling," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 2020, pp. 1678–1687.

[14] Z. Chen, X. Long, Y. Wu, L. Chen, J. Wu, and S. Liu, "Data aggregation aware routing for distributed training," in *Parallel and Distributed Computing, Applications and Technologies: 21st International Conference, PDCAT 2020, Shenzhen, China, December 28–30, 2020, Proceedings 21*. Springer, 2021, pp. 241–250.

Authorized licensed use limited to: Zhejiang Lab. Downloaded on August 03,2025 at 03:31:12 UTC from IEEE Xplore. Restrictions apply.